Clear Face Detection API

Technical Notes: Design, Published Gates, and the Closed-Eye Study

Abdullah Khalifa

4 November 2026

Version 1.0.0 | branch `fix/p0-detection` | HEAD `9459f86` | LaTeX 13 pp.

Abstract. The service takes a single base64-encoded image and decides whether the face in it is good enough to *enroll*. It is a gatekeeper: it returns a pass or fail verdict with a reason, not an identity. This note has three parts: (i) the pipeline and the gates it publishes, (ii) the correctness defects found and fixed while the API and the webcam page were made to fit it, and (iii) the closed-eye check built on landmark geometry, with the data behind it – and the claim that data does *not* support.

Contents

1 Problem Statement and Scope **3**
1.1 The request and the response contract . . . 3

2 Architecture **4**

3 Detection and Alignment **4**
3.1 YuNet on a fixed canvas . . . 4
3.2 The five landmarks . . . 5
3.3 Roll, spacing, and containment . . . 5

4 The Published Gates **5**

5 The Closed-Eye Problem **6**
5.1 Why the obvious check is not the check . . . 6
5.2 The licence gate comes before the model gate . . . 6
5.3 The Eye Aspect Ratio . . . 7
5.4 Two operating points . . . 7

6 Evidence: Two Corpora, Two Questions **7**
6.1 The measured distributions . . . 8
6.2 The price of each operating point . . . 8
6.3 The margin . . . 9

7 The Live Webcam Path **10**
7.1 One click, one stream . . . 10
7.2 Data URL, not a second encoder . . . 10
7.3 The overlay is drawn on the picture . . . 10

8 Defects Found and Fixed **10**
8.1 The concurrency race, in detail . . . 11

9 What This Study Does Not Claim **12**

10 Reproduction **12**

Appendix A: Threshold trade-off **13**

1 Problem Statement and Scope

The service accepts an image and decides whether the face in it may be used *for enrollment*. Enrollment is stricter than recognition: a recogniser will happily keep a blurry identity vector that happens to match the right person, but an enrollment system that stores that vector poisons every later match. The gate must therefore fail *early* and *legibly*.

The contract has three parts:

1. **Verdict.** `accepted` is a boolean, `flaws` a human-readable reason, and `checks` a list of per-gate records. A rejection without the failing gate's name is a bug, not a user-error message.
2. **No identity.** No recognition runtime is imported anywhere in the package. See Table 3 for why this is a licensing constraint, not a taste.
3. **One face.** Zero faces is *no face*; two or more is *ambiguous*. Both are rejections with their own text, never a silent pick of the largest box.

Note. The gate is not a "good photo" filter. A passport photo with dramatic shadows can pass; a bad webcam frame with three faces cannot. Every check answers *one* question about whether the extracted face geometry can be trusted.

Figure 1: The enrollment page. The detector's box and five keypoints are drawn over the same image it analysed.

1.1 The request and the response contract

The API takes JSON, not multipart. This is deliberate: the webcam path already holds a base64 data URL in memory, and multipart would mean the page re-encodes it into a second format for no gain. The request body is

```
{ "image": "<base64 data, optionally a data: URL>" }
```

The body is capped at ten mebibytes *after* decoding (`MAX_IMAGE_BYTES`). The response always carries the same keys, including when the request itself was malformed:

```
{
  "accepted":            bool,
  "flaws":               str,
  "face_detected":       bool,
  "face_count":          int,
  "image_size":          {w, h},
  "facial_area":         {x, y, w, h},
  "landmarks":           [[x, y], ... 5 points],
  "detection_confidence": float,
  "quality":             {...},
  "checks":              [ {"name", "passed", "value", "detail"}, ... ]
}
```

Why the contract is a stability requirement. The page renders its banner from `accepted`, its gate table from `checks`, and its overlay from `facial_area` and `landmarks`. Renaming one key is invisible in the server's own tests and silently breaks the client: the banner stays red forever and the overlay draws nothing.

2 Architecture

The package is small enough to read in one sitting, which is the point: every module answers one question.

Table 1: Module responsibilities.

Module	Responsibility
`app/main.py`	FastAPI surface: request model, JSON validation, static files, and the one error handler that keeps the response shape.
`app/_detector.py`	Load and checksum the YuNet ONNX blob; letterbox-to-`DET_SIZE`; *serialise* `setInputSize` and `detect`; map boxes back to source pixels.
`app/_align.py`	Five-point alignment: roll angle, spacing ratios, "landmarks inside" tests.
`app/_eyes.py`	Closed/open-eye gate on MediaPipe FaceLandmarker mesh points (Section 5).
`app/face_service.py`	Orchestration: decode, detect, crop gates, assemble `Check` list, build payload.
`static/index.html`	Upload page and live webcam loop (Section 7).
`bench/eye_bench.py`	Evidence-set builder: one row per image, both corpora.
`bench/eye_stats.py`	The numbers behind the eye threshold.

Invariant. `face_service` is the only module that decides *pass or fail*. The detector returns boxes and keypoints; the aligner returns angles and ratios; the eye module returns a measurement. Turning measurements into a verdict happens in one place.

3 Detection and Alignment

3.1 YuNet on a fixed canvas

The detector is YuNet shipped inside OpenCV (`cv2.FaceDetectorYN`, Apache-2.0). The ONNX blob is not vendored in the repository: `python -m app.fetch_models` downloads it and `_detector.py` refuses to load a file whose SHA-256 differs from the pinned digest. A swapped blob is therefore either a build failure or an explicit edit, never a silent behaviour change.

YuNet is loaded with a fixed input size

$$\text{DET_SIZE} = (640,\, 640), \tag{1}$$

so an arbitrary image must be letterboxed first. Let (W, H) be the source size and

$$s = \min\left(\frac{640}{W}, \frac{640}{H}\right) \tag{2}$$

the scale that fits it. The resized image is centred with pads

$$p_x = \left\lfloor \frac{640 - Ws}{2} \right\rfloor, \qquad p_y = \left\lfloor \frac{640 - Hs}{2} \right\rfloor, \tag{3}$$

and every returned box is mapped back to source pixels by

$$\text{src} = \frac{\text{canvas} - p}{s}. \tag{4}$$

The letterbox exists so that a landscape frame and a portrait frame are *judged at the same effective resolution*; without it the detector's pixel budget depends on the upload's aspect ratio.

3.2 The five landmarks

The five keypoints are right eye, left eye, nose, right mouth corner and left mouth corner, *as seen by the image*, i.e. they are not mirrored to a person-centric frame. Every alignment formula below is therefore written for the image frame, and the page draws them in the same order it receives them.

3.3 Roll, spacing, and containment

Definition (roll). *The roll of the face is the angle of the eye line against the image horizontal:*

$$\theta = \operatorname{atan2}(y_L - y_R,\ x_L - x_R)\,. \tag{5}$$

A frame passes when $|\theta| \le \text{MAX_ROLL_DEG}$, *with* $\text{MAX_ROLL_DEG} = 15°$.

Definition (five-point mean). *For points* $p_1, \ldots, p_5$, *the centroid is*

$$\bar{p} = \frac{1}{5}\sum_{i=1}^{5} p_i. \tag{6}$$

It is used as the reference the five points are tested against: a landmark that falls outside the face box while the centroid is inside is a corrupted keypoint, not a tilted head.

Why the centroid and not the box centre. A tilted or turned head moves the box centre away from the mesh; the centroid moves with the mesh. Comparing points to the box centre would reject lawful 3/4 poses, so the centroid is the reference and the box only supplies the bounds.

Why five points, not 68 or 478. Five points are the smallest set that still fixes *pose*: two eyes give roll, and the nose and both mouth corners give pitch and yaw roughly. A dense mesh fixes shape as well, but shape is identity – the very thing a quality gate must not depend on.

4 The Published Gates

Every gate emits a `Check` record, so the page can name the failure instead of showing red. The thresholds are constants in `app/_detector.py` and `app/face_service.py`.

Table 2: Gate summary. *Value* is what the record reports to the client.

Gate	Threshold	Verdict condition and reported value
`one_face`	exactly 1 face	pass iff one box after confidence filtering; value: count
`face_size`	$\ge 80 \times 80$ px	pass iff $w \ge 80$ *and* $h \ge 80$; value: "wxh"
`detection_confidence`	≥ 0.80	pass iff det score reaches it; value: score to 3 dp
`blur`	var ≥ 50	Laplacian variance of the crop, resampled; value: variance
`exposure`	$35 \le b \le 225$	pass iff mean grey of crop is in band; value: b
`head_pose`	$\lvert\theta\rvert \le 15°$	pass iff roll within limit; value: degrees
`landmarks_inside`	centroid test	pass iff all five points sit inside the box
`landmark_spacing`	ratio band	pass iff inter-point distances are plausible

Pitfall: blur is resolution, not focus. The Laplacian variance of a *fixed metric* on a fixed face scales with the pixel count of the crop. The same face scores 506 at 512 px and 12 at 1600 px. A raw cutoff therefore judges the camera, not the sharpness of the frame. The crop is resampled to BLUR_REFERENCE_WIDTH = 112 px before the variance is taken, so the number is comparable across uploads.

Zero-error edges. A crop or a landmark set that lies on a boundary (for example a face of exactly 80×80 px) is accepted, not rejected: the rules use $\geq$ and $\leq$. An empty crop is a rejection with its own message, and is never measured as "variance zero, therefore blurry": that would report a confident but false reason.

5 The Closed-Eye Problem

5.1 Why the obvious check is not the check

The repository contained a plausible left-eye/right-eye and "are both eyes open" routine already. It was wrong by construction. The routine measures a horizontal gradient ratio and concludes "open" when the eye region is *edge-rich*. A closed lid is precisely an intense horizontal edge, so the ratio peaks exactly on the frames that must be rejected. The wrong answer is not a side effect of a bad threshold; it is what the feature rewards. That module was unpicked in Section 8 and was withdrawn from the path of the gate.

The failure mode to keep in mind. A heuristic that is wrong in the same situation it is supposed to detect cannot be tuned into correctness. It must be removed, not re-thresholded cleanly.

5.2 The licence gate comes before the model gate

The requirement was an asset that can ship in a commercial product: open licence, commercially usable, integrated and testable. That requirement eliminated the obvious choices before any of them was trained:

Table 3: Datasets and models considered for the closed/open decision.

Asset	Real faces	Usable in a shipped product?
CEW (NUAA), 2 423 faces	yes	**No.** Research use only, non-commercial, no redistribution.
HF `closed-open-eyes`	mixed	**No.** Its closed set is CEW under another name; its open set is a subset of the 200 k `celeba` Hugging-Face images, unlabelled.
MRL Eye, ZJU	eye crops	**No.** Not whole faces, so not the input of this pipeline.
`face_landmarker.task` (MediaPipe)	—	**Yes.** Apache-2.0, 3 758 596 bytes, float16, no eye-state label.

Consequence. The gate cannot be a learned open/closed classifier trained on the datasets that carry that label, because none of them may travel with the product. The remaining road is to *derive* eye state from an asset that is allowed to ship.

5.3 The Eye Aspect Ratio

The landmarker returns a mesh of N points ($N = 468$, or 478 with the iris refinement). Two contours of six points describe each eye:

$$\ell = (p_{33},\, p_{133},\, p_{159},\, p_{145},\, p_{158},\, p_{153})\,, \quad r = (p_{362},\, p_{263},\, p_{386},\, p_{374},\, p_{385},\, p_{380})\,. \tag{7}$$

Write a contour as $(c_1, c_2, u_1, l_1, u_2, l_2)$: two corners, then upper and lower lid points. The Eye Aspect Ratio is

$$\mathrm{EAR} = \frac{\|u_1 - l_1\| + \|u_2 - l_2\|}{2\,\|c_1 - c_2\|}. \tag{8}$$

Equation (8) is a ratio of *lengths on the same face*, so it is invariant to the picture's scale, to the camera's distance, and (up to foreshortening) to translation. This is the property that makes a fixed threshold legitimate.

The assumption that breaks it: yaw. Under a strong head turn the corner distance shrinks by $\cos\varphi$ while the lid distances barely do, so the EAR *rises*. A closed eye in profile can therefore read as open. The eye gate must not be trusted on faces that are turned far to the side; it is a companion to `head_pose`, not a replacement for it.

5.4 Two operating points

A single cutoff makes verdicts alternate. On a camera, the frame after a blink lands on the other side of the cutoff; a JPEG smears the lid edge, and the next poll flips the answer. The gate uses a band:

$$\begin{cases} \text{closed}, & \text{if } \mathrm{EAR} < 0.18, \\ \text{hold (half closed or blinking)}, & \text{if } 0.18 \le \mathrm{EAR} < 0.20, \\ \text{open}, & \text{if } \mathrm{EAR} \ge 0.20. \end{cases} \tag{9}$$

The middle band rejects by asking the person to hold still, not by accusation "eyes closed". On a polling camera the median of the last K frames is what the decision uses (Section 7), so one blink cannot end a session.

Note: not measurable is not closed. Below a corner distance of 20 px the lid is one or two pixels thick, and the ratio is noise. The module returns `None`, and the gate says *"Eyes could not be measured in this frame"*. Asserting *closed* there would turn a detection failure into a rejection and the eye test into a single point of failure.

6 Evidence: Two Corpora, Two Questions

A single "accuracy" for this gate would be a fabrication, because the two corpora that were run answer different questions and only one of them carries an eye-state label.

Table 4: What each corpus can and cannot support.

Corpus	Images	Label	Permitted claim
CEW subset (via HF set)	400	**closed**	detection rate: share of *closed* eyes rejected
CelebA (shards 0–1)	3 070	none (eye state)	behaviour: pass rate, i.e. the false-reject cost

The Hugging-Face set `MichalMlodawski/closed-open-eyes` presents itself as balanced closed/open. Its `closed_eyes` rows are CEW faces; its `open_eyes` rows come from a different upload and are a subset of CelebA, where CelebA has no eye-state column. A combined accuracy over that dataset would therefore measure on a target that was invented by the packaging. The evidence set keeps the two corpora *separate* and carries the corpus name in every row.

Figure 2: Eye Aspect Ratio of the two corpora, with the two operating points. The overlap between the distributions is the entire cost of using a fixed threshold.

Figure 3: Passing share against threshold. The closed corpus falls to near-zero; the unlabelled corpus has no true eye label caps, so its curve is a lower bound on false rejects only.

6.1 The measured distributions

For $n = 400$ labelled closed images and $n = 3\,049$ measurable unlabelled images of the 3 070, the per-image estimate used by the gate is

$$\hat{e} = \min(\mathrm{EAR}_\ell,\ \mathrm{EAR}_r)\,, \tag{10}$$

i.e. the *more closed* of the two eyes decides. The sample percentiles of $\hat{e}$ are the numbers behind Figures 2 and 3:

Table 5: Percentiles of $\hat{e}$ per corpus (measured, not taken from the papers). The CEW side is 100 % labelled closed.

Corpus	n	p_1	p_5	median	p_{95}	max
CEW closed	400	0.005	0.007	0.034	0.116	0.308
CelebA	3 049	0.085	0.177	0.309	0.387	0.571

6.2 The price of each operating point

Define the rejected share at threshold t as

$$R(t) = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}[\hat{e}_i < t]\,. \tag{11}$$

For the labelled CEW corpus $R(t)$ is a *miss* rate (closed faces accepted); for the unlabelled CelebA corpus it is *at best* an upper bound on the false-reject rate, because an unknown share of those faces are genuinely dozing, blinking, or squinting and the gate rejects them *correctly*.

Table 6: Rejected share $R(t)$ per corpus at candidate thresholds. A CelebA row is a cost, not an accuracy: it counts rejections, not errors.

t	CEW closed rejected	CelebA rejected
0.10	91.75 %	1.41 %
0.12	95.25 %	2.10 %
0.15	97.50 %	3.64 %
0.18	98.75 %	5.28 %
0.20	99.00 %	6.76 %
0.22	99.50 %	9.35 %
0.25	99.75 %	17.71 %

Reading the table honestly. At $t = 0.20$, 6.76 % of CelebA frames are rejected. Were every one of them open, the echo would be the false-reject rate; but the corpus was photographed in the wild and some of those faces are genuinely asleep-eyed, so the measured 6.76 % is an *upper bound* on the false-reject rate, not the false-reject rate itself, and the true false-reject rate is smaller. Until the unlabelled corpus is manually graded, the gate can quote detection on the closed side and *only a cost bound* on the open side.

6.3 The margin

Let $\mathrm{EAR}^{\mathrm{cl}}$ and $\mathrm{EAR}^{\mathrm{op}}$ be the random variables behind the two corpora. The band in Section 5 is defensible exactly when

$$\sup_{F_{\mathrm{cl}}} t < \inf_{F_{\mathrm{op}}} t, \qquad \text{but no gap was measured here.} \tag{12}$$

The closed corpus has 95th percentile 0.116 and max = 0.308; the unlabelled corpus has 5th percentile 0.177 and min = 0.012. Their percentile ranges *cross*. A band placed anywhere between them trades a visible closed-side miss for an unseen open-side reject; the chart makes that trade visible, and no wording should make it invisible.

The claim this study may not make.

1. "Accuracy of X %" over the union of the two corpora: the unlabelled corpus has no eye-state ground truth.
2. A closed-side miss rate under any rounding that folds the 0.18–0.25 tail into "noise": those are 5 of 400 faces at $t = 0.20$, so the honest miss rate is 1.25 %, and rounding it to "0 %" would be a false claim on five faces.
3. Anything resting on a single image: the 0.20–0.25 band holds four images, and one of them is more than half of the residual mass.

Scope. The numbers above are from *still JPEG frames*, mostly `178` × `218` px crops or similar. The webcam path is live, motion-blurred, and polled every 900 ms. Neither distribution transfers to the camera for free; a camera-side acceptance must be measured again on camera frames, not on these images.

7 The Live Webcam Path

7.1 One click, one stream

The earlier page sent a single frame per click. That made enrollment a matter of timing luck: the operator had to press the button exactly on a still, open-eyed frame. The current page opens the camera once, then posts a frame on a timer:

$$\text{CAM_POLL_MS} = 900 \text{ ms}, \tag{13}$$

while three guards keep the loop from turning into feedback:

1. `camBusy` – a frame is in flight, so the next tick is skipped, not queued. On a two-core host a queue would only build latency.
2. `lastDataUrl` – the canvas encoded the same frame as last time (camera asleep, unchanged scene), so nothing is posted.
3. *black-frame hold* – the first polls after the stream opens return a black canvas because the sensor has not warmed up; those frames are not posted.

The stream stops on the second press, on page unload, and on the first error. A failed frame never leaves a polling interval running.

7.2 Data URL, not a second encoder

The camera path holds a base64 data URL in memory, while the upload path holds a `Blob`. `verify()` therefore accepts both: for a string it slices after the first comma, for a blob it reads the file. The failure this replaced was reported as

```
FileReader.readAsDataURL: parameter 1 is not of type 'Blob'.
```

and was not a camera bug: the camera was correct; the consumer simply expected the upload's type. The fallback in `showImage()` has the same shape – a string is passed straight to `img.src`, a blob becomes an object URL.

7.3 The overlay is drawn on the picture

The page draws the detector's box and its five keypoints onto a canvas that sits *on* the displayed image. The coordinate space is

$$\text{canvas px} = \text{CSS px} \times \text{devicePixelRatio}, \tag{14}$$

so a canvas of size $w \times h$ CSS pixels is backed by $(w \cdot d) \times (h \cdot d)$ pixels while its style stays $w \times h$. Drawing in CSS units and multiplying the transform by d keeps a $2\times$ screen and a $1\times$ screen geometrically identical. The canvas was earlier absolutely positioned with no positioned ancestor, so it escaped the image box; `#boxStage` now carries `position: relative`, and the zoom/pan transform is applied to the image only, never to the canvas's coordinate space. The measured misalignment is 0 px on all four edges.

8 Defects Found and Fixed

Each item below was reproduced before it was fixed; the "signature" column is the observation that would have identified it from the outside.

Table 7: Defects fixed on `fix/p0-detection`.

Area	Signature	Cause and fix
Upload request	HTTP 400, *Invalid JSON request*	Page posted `FormData`; endpoint reads a JSON `image`. Post JSON base64 instead.
Banner	always red	Page read `acceptable_for_enrollment`; the field is `accepted`.
Gate table	every row "N/A"	API did not return `checks`; added to the response.
`exposure`	green on relative dark	A missing brightness counted as passing. Missing is a skip, not a pass.
Overlay	box beside the image	Canvas had no positioned ancestor; `#boxStage` made `absolute/relative` box.
Camera	*not of type 'Blob'*	`verify()` assumed a Blob; camera holds a data URL. Accept both types.
Camera (HTTP)	500 under concurrency	`setInputSize` + `detect` raced on one stateful session. Both now under `_DETECTOR_LOCK`.
Eyes (dead)	closed eyes passed	`app/quality.py` is imported by nothing; its edge ratio peaks on the closed lid and it was never in the pipeline.

The eye check was never wired in. The page, the endpoint and the "eyes" prose existed, but no call site reached the eye test: the live checks were `one_face`, `face_size`, `detection_confidence`, `blur`, `exposure`, `head_pose`, `landmarks_inside` and `landmark_spacing`. Painting both eyes black on the test portrait returned `accepted: true`. The diagnosis is therefore a *missing call site*, not a mistuned threshold; no value of a cutoff would have changed the result.

8.1 The concurrency race, in detail

YuNet's `FaceDetectorYN` keeps the input size of the last call as session state in the C++ object. With this code,

```
detector.setInputSize(DET_SIZE)   # writes to the session
faces = detector.detect(...)      # reads the session's size
```

two worker threads can interleave as **A set** → **B set** → **A detect**, and A decodes B's size. The observed rate on four uvicorn workers with eight parallel 1280 × 720 frames was four or five of eight requests failing with `500 analysis_failed`. Because the decode step is the expensive one, it stayed outside the lock; only the `setInputSize` and `detect` pair moved inside it. Result: 8/8 then $3 \times 8/8 = 24/24$ successful, and the serialised detection is per request, so a single face per frame adds no visible latency.

9 What This Study Does Not Claim

Claims that a reader may want and that the evidence above does not support.

- *"The eye gate is X % accurate."* The labelled corpus carries only the closed label. Accuracy over the union would repeat the packaging error of Section 6.
- *"The CelebA pass rate is the open-eye acceptance rate."* CelebA has no eye-state column, and an unknown share of its faces are genuinely not open-eyed, so the pass rate bounds the false-reject rate *from below*, not from above.
- *"The still-frame calibration transfers to the webcam."* These corpora are small still crops; the live path is full-frame, motion-blurred, and polled at 900 ms. It must be measured on camera frames.
- *"Eyes closed at any angle."* Under a strong yaw the ratio rises (Section 5); profile faces are outside the contract of this gate and the `head_pose` check is what refuses them.

Current posture before the open-side grading. The gate rejection it emits is correct on the closed side (98.75 % at 0.18 on real CEW faces) but its false-reject cost on the open side is only *bounded*. That is enough to ship a narrow, conservative gate that holds rather than offends; it is not enough to print "accuracy" in a product page.

10 Reproduction

The two scripts under `bench/` are the whole pipeline behind Section 6. They are deliberately small and have no hidden state.

```
# 1. One row per image, both corpora, every gate decision recorded.
python bench/eye_bench.py \
    --out ~/.hermes/cache/scratch/celeba/eye_rows.jsonl

# 2. The percentiles behind the tables, read back from (1).
python bench/eye_stats.py \
    ~/.hermes/cache/scratch/celeba/eye_rows.jsonl
```

A row looks like this – the corpus field is what keeps the two questions apart downstream:

```
{"corpus": "cew_subset", "source": "eyes_001", "row": 0,
 "label": "closed", "sha256": "9f0c...", "height": 218, "width": 178,
 "eyes_ok": false, "measurable": true,
 "reason": "Eyes appear closed.", "ear_l": 0.041, "ear_r": 0.038,
 "corner_px": 33.2, "ear": 0.038}
```

The row is a measurement, not a verdict carrier. The gate decision is stored *alongside* the number (`ear_l`, `ear_r`) and the corner distance, so a threshold can be re-chosen later without re-running the landmarker. Re-deriving a table from stored `eye_rows.jsonl` is a one-line numpy call; re-running the model is two minutes of CPU.

Determinism caveat. MediaPipe is not bit-reproducible across its own versions, and the rows carry the model's SHA-256 (`64184e22...`) for that reason. A free-floating percentile does not move much under a patch bump; a threshold placed *inside* a dense band (which is exactly where 0.18 and 0.20 sit) does.

Appendix A: Threshold trade-off, in one table

Table 8: The shipped operating point $t = 0.18$ against its neighbours.

t	CEW closed rejected	CelebA rejected	Direction of the move
0.12	95.25 %	2.10 %	catches 1 closed in 21; large leniency
0.15	97.50 %	3.64 %	the balanced point
0.18	**98.75 %**	**5.28 %**	**shipped: claims more, costs roughly** +45 % **rejections**
0.20	99.00 %	6.76 %	+1 face of 400, +28 % cost

The last column is the honest way to read the third: between 0.15 and 0.18 the gate buys one closed face in 400 (97.50 → 98.75) and pays roughly a quarter more in rejects (3.64 → 5.28, i.e. +45 % on the count of rejections). That is a small buy for a visible price; it must be sold as such, not as “improved accuracy.”